AI Model Building

Transformer-Based Sentiment Classification: Training, Evaluation, Optimisation, and Deployment

Assignment: AI Model Building

Prepared on: 29 October 2026

Research report and practical implementation guide

Abstract

This report presents an end-to-end AI model-building project using Transformers for binary sentiment classification. It covers dataset preparation, tokenization, fine-tuning, evaluation, optimisation, and deployment. The project uses DistilBERT, a compact Transformer encoder, and the Rotten Tomatoes review dataset. It also discusses limitations, responsible use, and how to record genuine experimental results.

1. Introduction

Transformers are neural-network architectures that use attention to model relationships between tokens. Pre-trained Transformer models learn general language representations from large corpora and can then be adapted to tasks such as sentiment classification, topic classification, question answering, and summarisation. Fine-tuning an existing model is generally more practical for a student project than training a large model from scratch.

2. Objective and Problem Definition

The objective is to build a model that classifies a short movie-review snippet as positive or negative. Input is review text; output is one of two sentiment labels. This is a supervised text-classification task because training examples include known labels. The project demonstrates a complete workflow rather than claiming to train a foundation model from scratch.

3. Dataset Preparation

The project uses the Rotten Tomatoes dataset through Hugging Face Datasets. It provides train, validation, and test splits. The script loads the data, tokenizes text using the matching DistilBERT tokenizer, truncates sequences to a maximum of 128 tokens, and passes the tokenized examples to the model. The validation split supports model selection; the test split is reserved for final evaluation. Check the dataset card and license before use beyond education.

4. Transformer Model Architecture

DistilBERT is a smaller, distilled variant of BERT designed to reduce model size and inference cost while retaining useful language representations. Text is converted into subword tokens. The Transformer encoder processes token representations using self-attention and feed-forward layers. A sequence-classification head maps the pooled representation to two class scores. A softmax distribution can be interpreted as the model's class confidence, though it is not a guarantee of correctness.

5. Training Protocol

The starter training script uses a learning rate of 2e-5, training batch size 16, evaluation batch size 32, maximum sequence length 128, two epochs, weight decay 0.01, and random seed 42. It evaluates on validation data during training, selects the best checkpoint by accuracy, evaluates the held-out test split, and saves the model and tokenizer. Hardware, library versions, and runtime can affect results; CPU training may be slow.

6. Evaluation Metrics

Accuracy measures the fraction of predictions that are correct. Weighted F1 combines precision and recall for each class and weights each class by its support. These metrics give an overview but do not explain every error. A stronger evaluation includes a confusion matrix, per-class precision and recall, and manual inspection of examples such as sarcasm, mixed opinions, or ambiguous wording. Record only the actual metrics printed by the script after it runs.

7. Model Optimisation

Potential experiments include varying the learning rate, number of epochs, batch size, sequence length, and model size. Compare changes using the validation split and retain the test split for final evaluation. If training and validation performance diverge, reduce overfitting with fewer epochs, weight decay, early stopping, or more suitable data. Use reproducible seeds and document each experiment.

8. Deployment Options

A command-line interface is simplest for a demonstration. Streamlit can provide a browser-based user interface, while FastAPI can expose a REST endpoint. Cloud deployment requires dependency pinning, model-artifact management, access controls, rate limiting, monitoring, and budget checks. The saved model can be large, so repository files should normally contain code and instructions rather than committed model weights; use an appropriate model-artifact store if needed.

9. Limitations and Responsible AI

Sentiment depends on context, sarcasm, cultural references, and domain-specific wording. Training data can contain biases and may not represent every user group. Confidence scores should not be treated as certainty. This educational classifier should not be used for high-impact decisions such as hiring, grading, credit, or healthcare. Inspect errors, explain limitations, and evaluate on data representative of the intended use.

10. Conclusion

This project demonstrates the main stages of Transformer model building: selecting a dataset, tokenizing text, fine-tuning a pre-trained model, evaluating held-out examples, saving artifacts, and running inference. DistilBERT provides a practical starting point for a student project. A complete submission should include the actual test metrics and observations from a successful run, not guessed scores.

References

[1] Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS. https://arxiv.org/abs/1706.03762\n\n[2] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. https://arxiv.org/abs/1910.01108\n\n[3] Hugging Face. Transformers documentation. https://huggingface.co/docs/transformers\n\n[4] Hugging Face Datasets. Dataset loading documentation. https://huggingface.co/docs/datasets